Prototype of an educational 3D videogame on Crystallography

GABRIELE PRINCIOTTA

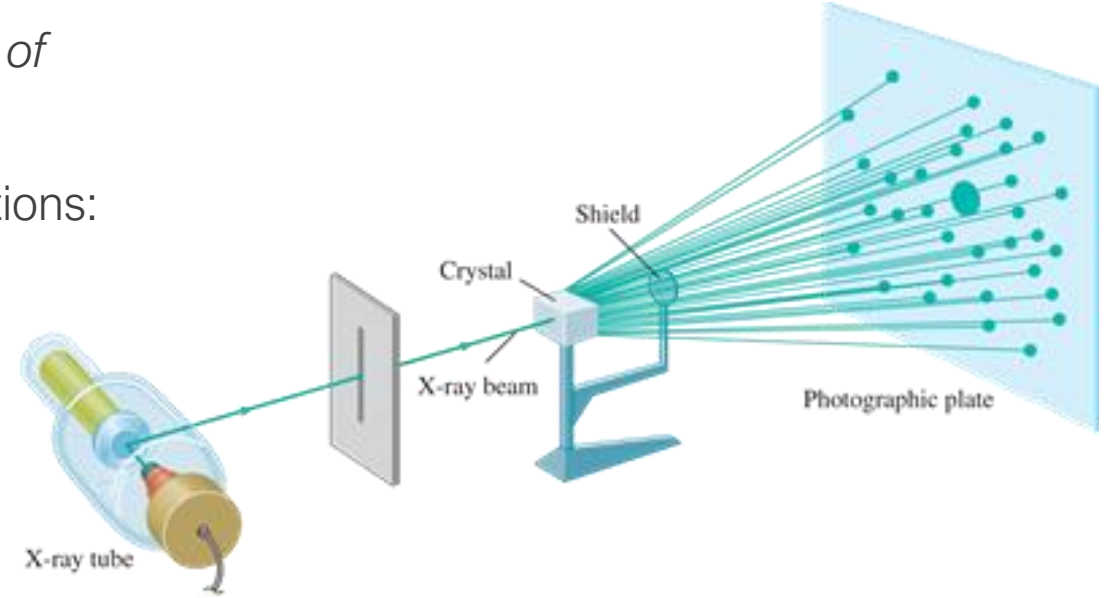
Supervisor: MARCO TARINI

Background: X-ray Crystallography

A technique to *reconstruct the 3D structure of molecules.*

Used in various research areas and applications:

- structural biology
- protein science
- analytical and synthetic chemistry

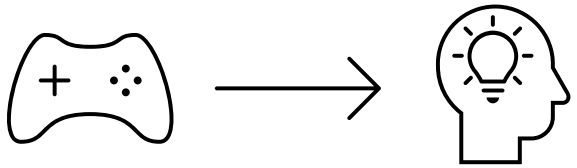


This project is promoted by the Italian Crystallographic Association (AIC).

Goal of the project

Creating an educational videogame on Crystallography:

- Game Design: make the player learn through game mechanics
- Software development: game mechanics implementation and creation of the necessary assets



Design a game with the goal of divulge Crystallography is a hard task!

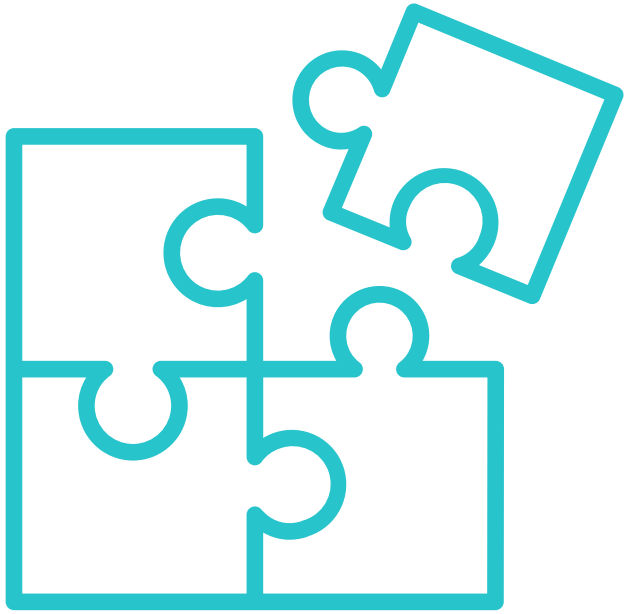
Development of a prototype to explore a gameplay hypothesis.

Eurographics

Eurographics Italian Chapter

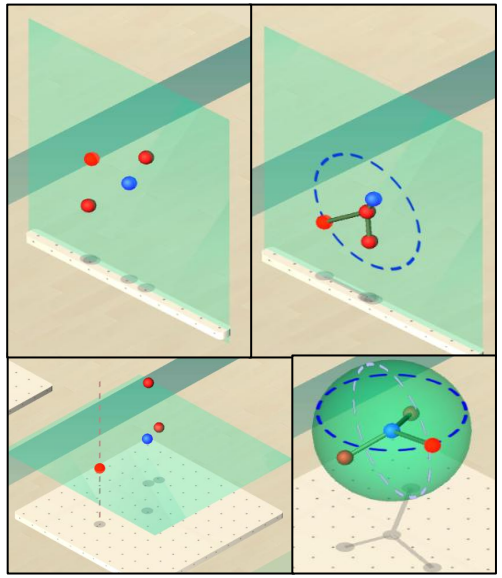
STAG

Gameplay hypothesis: Puzzle Game

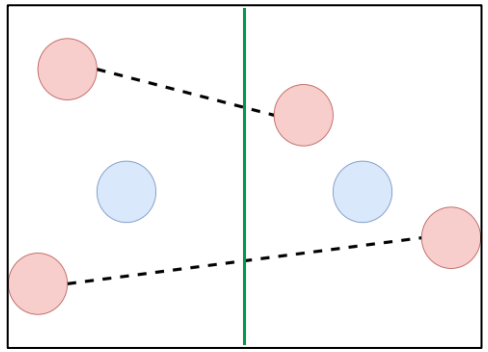


Let's see a playable demo:

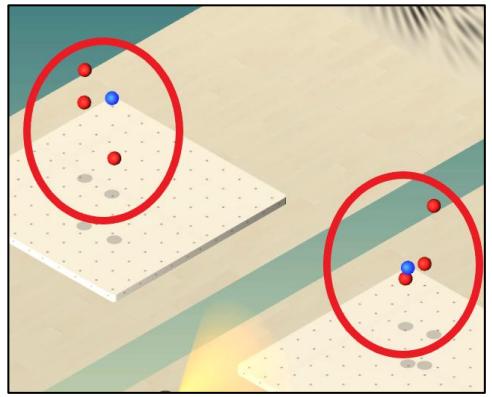
https://tinyurl.com/CrystalloGame



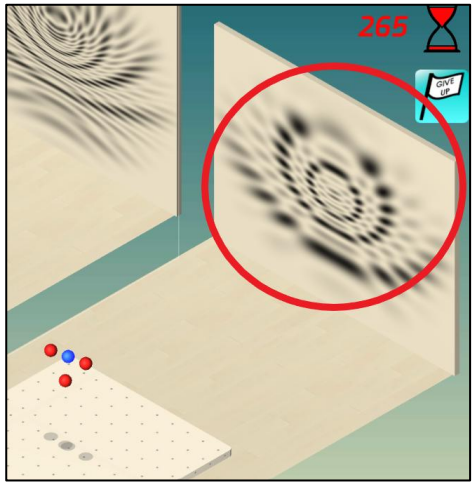
GUI for controlling the atoms in 3D



Solution recognition algorithm



Random levels generation

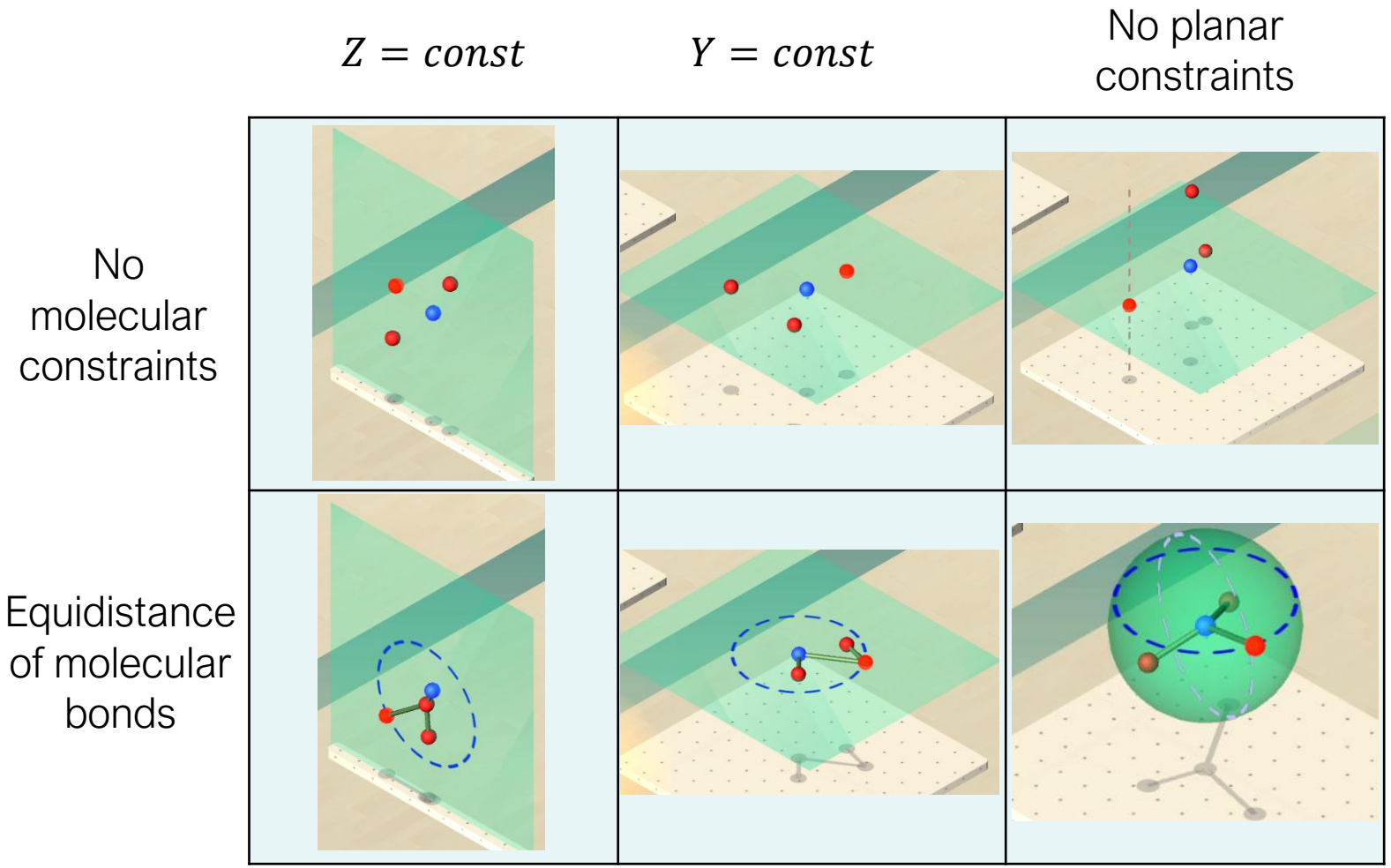


Real-time synthesis of diffraction pattern

Main technical challenges faced

GUI for controlling the atoms in 3D

- 3D position: communicated with vertical shadows and other graphical aids
- Drag and drop (ray tracing implementation)
- 3D gizmos diversified according to the active constraints



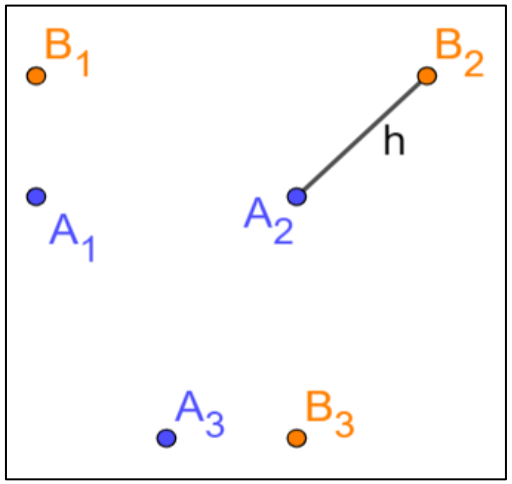
Solution recognition algorithm

Hausdorff distance was used as a parameter to verify the similarity between two sets of positions (x, y, z).

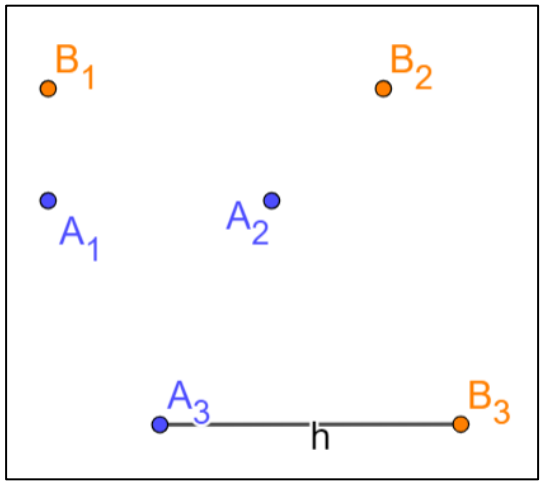
Hausdorff distance:

maximum distance of a set to the nearest point in the other set, that is:

$$h(\mathrm{A}, \mathrm{B}) = max_{a \in A} \{min_{b \in B}\{d(a, b)\}\}$$



$h(A, B) = 0.7$



$h(A, B) = 1.35$

Random levels generation

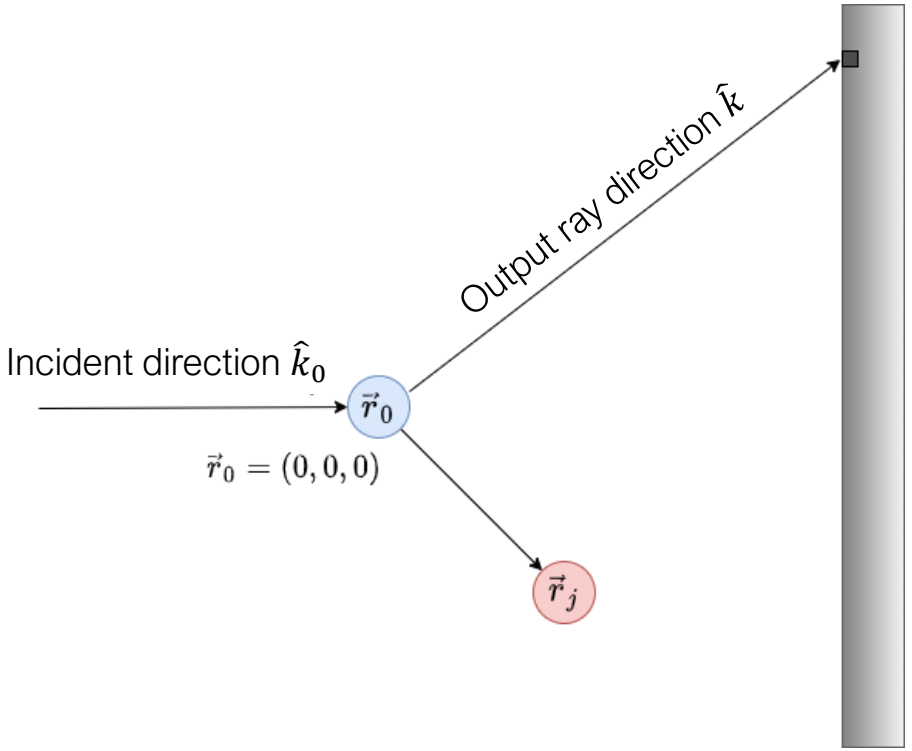
	Solution generation	Initial state generation
Molecule mode NOT active	1. Random uniform distribution	1. Random uniform distribution
Molecule mode active	1. Random uniform distribution 2. Prim's algorithm MST	1. Random uniform distribution 2. Recursive setting of equidistance constraint

Synthesis of diffraction pattern: problem formulation

$\varphi_j = -2\pi\vec{r}_j \cdot \vec{s}$ Contribution of the j-th atom

$\vec{s} = \dfrac{\hat{k} - \hat{k}_0}{\lambda}$ Value which defines the detector point of which we are calculating the intensity

$\left|\sum_{j=0}^{n-1} e^{i\varphi_j}\right|^2$ Intensity of the detector point defined by vector $\vec{s}$



Synthesis of diffraction pattern: implementation (1)

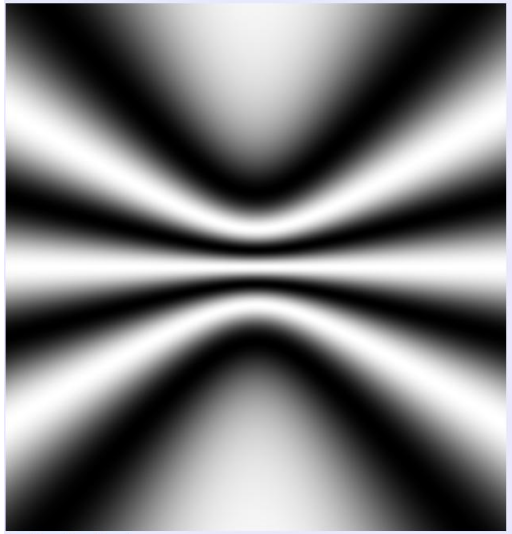
Eurographics

Eurographics Italian Chapter

STAG

1. Formula implemented in a **GLSL** (OpenGL) fragment shader, then translated in **HLSL** (DirectX) to be used inside Unity

GLSL sandbox (GID - UniMi 2018/2019)



```
vec3 ks(vec2 screenCoords)
{
    float zoom = 4.0;
    return normalize(vec3(screenCoords*zoom, 1.0));
}

vec2 e_pow_ix(float x)
{
    return vec2(cos(x), sin(x));
}


void mainImage( out vec4 pixelColor, in vec2 pixelCoord )
{
    // normalize pixel coordinates (from 0 to 1)
    vec2 screenCoords = 2.0 * (pixelCoord/iResolution.xy - vec2(0.5));
    const vec3 k0 = vec3(1,0,0);
    const float lambda = 0.5;

    vec2 I = vec2(1.0,0.0);
    vec3 s = (ks(screenCoords)-k0)/lambda;
    vec3 r1 = vec3(0,1,0);
    float phi1 = -2.0*3.1415*dot(r1,s);
    I += e_pow_ix(phi1);

    pixelColor = vec4(dot(I,I) / 4.0);
}
```

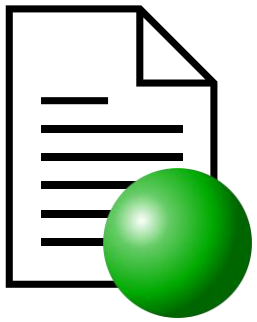
HLSL

```
half2 e_Pow_ix(half x)
{
    return half2(cos(x), sin(x));
}


half4 frag(v2f_customrendertexture IN) : SV_Target
{

    const half2 screenCoords = 2.0 * (half2(IN.globalTexcoord.x, IN.globalTexcoord.y) - half2(0.5,0.5));
    int i;
    half2 I = half2(1.0, 0.0);
    half3 s = (ks(screenCoords)-k0)/lambda;
    for (i = 0; i < n_atoms; i++) {
        phi[i] = -2.0 * PI * dot(atomsPos[i], s);
    }
    for (i = 0; i < n_atoms; i++) {
        I += e_Pow_ix(phi[i]);
    }

    return half4(1,1,1,2) -
            half4(dot(I,I) / ((n_atoms+1)*(n_atoms+1)),
                  dot(I,I) / ((n_atoms+1)*(n_atoms+1)),
                  dot(I,I) / ((n_atoms+1)*(n_atoms+1)),
                  1);

}
```

Synthesis of diffraction pattern: implementation (2):

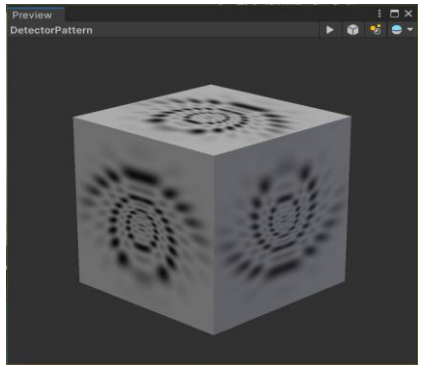
Eurographics Italian Chapter

2. Usage of a Unity *Custom Render Texture* (CRT)



Shader given to the CRT

CRT produced in real-time by the shader



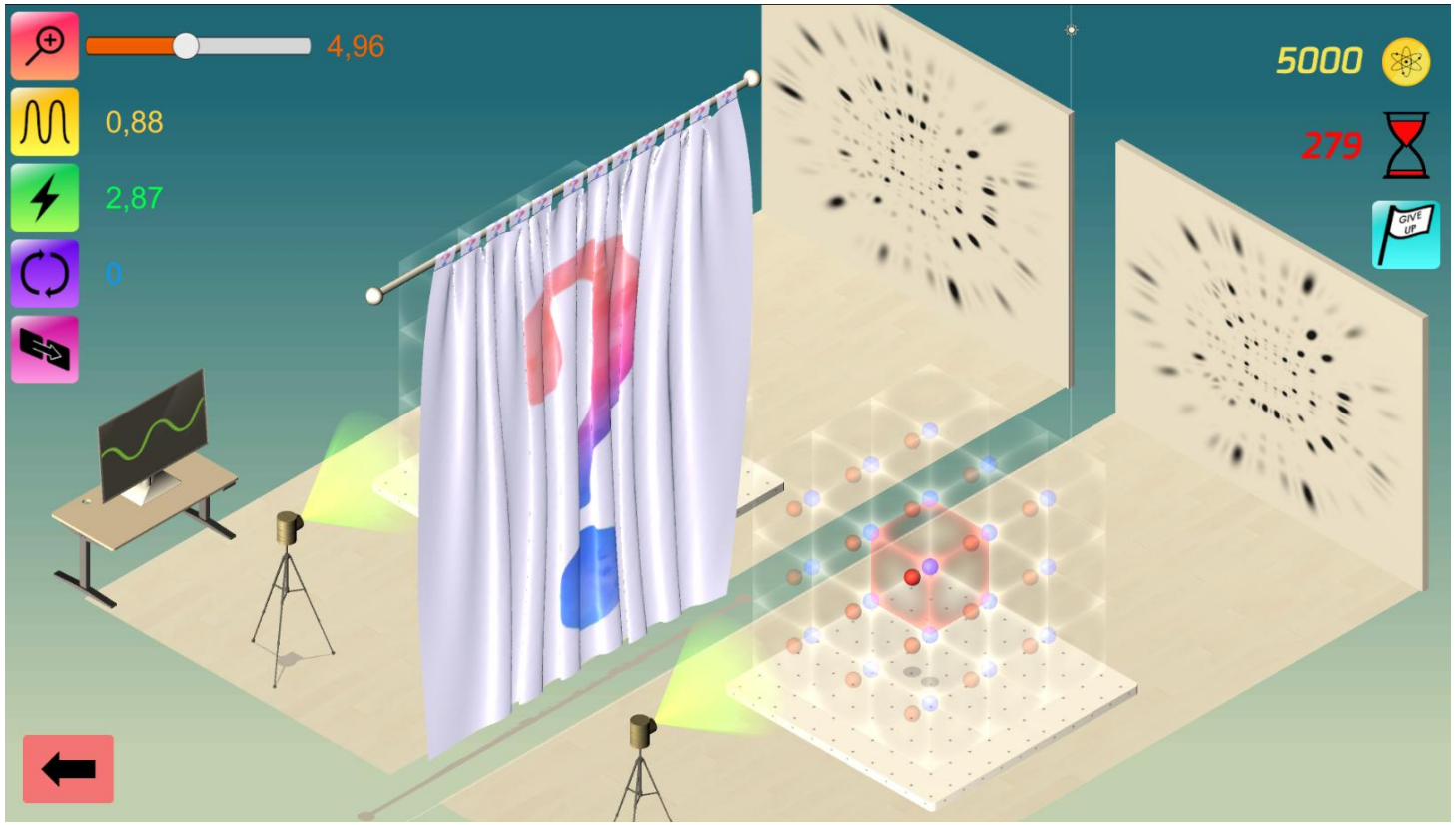
Material with the CRT applied on

3. Data stream to the shader – `uniform` variables

Crystal mode

X-ray Crystallography uses crystals as samples to improve the effectiveness of the diffraction phenomenon.

We tried to simulate this situation in the game.



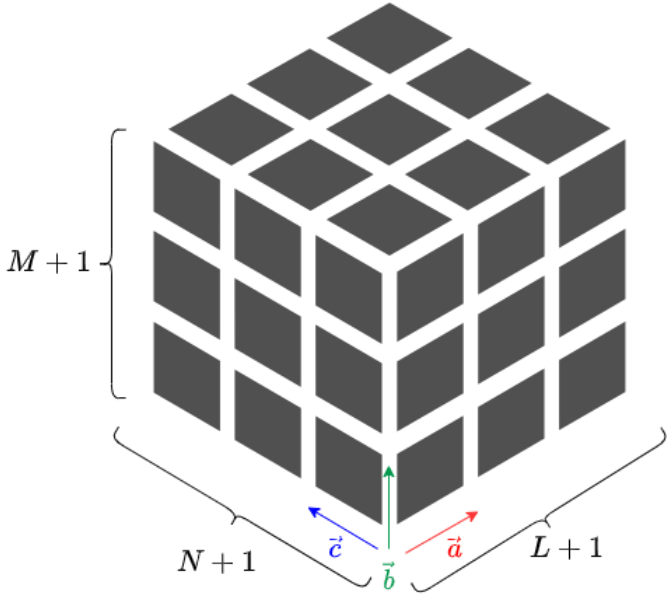
Synthesis of diffraction pattern: crystal multiplicative coefficient

Eurographics Italian Chapter

It is a **closed form** expression:
it scales very well with the repetitions number!

$$\left|\sum_{j=0}^{n-1} e^{i\varphi_j}\right|^2 \cdot K$$

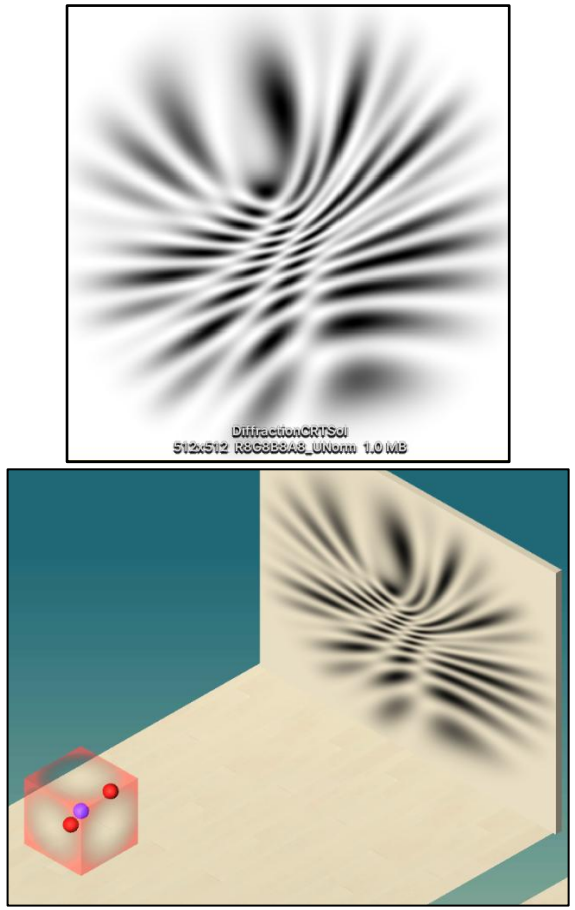
Intensity considering crystal repetitions



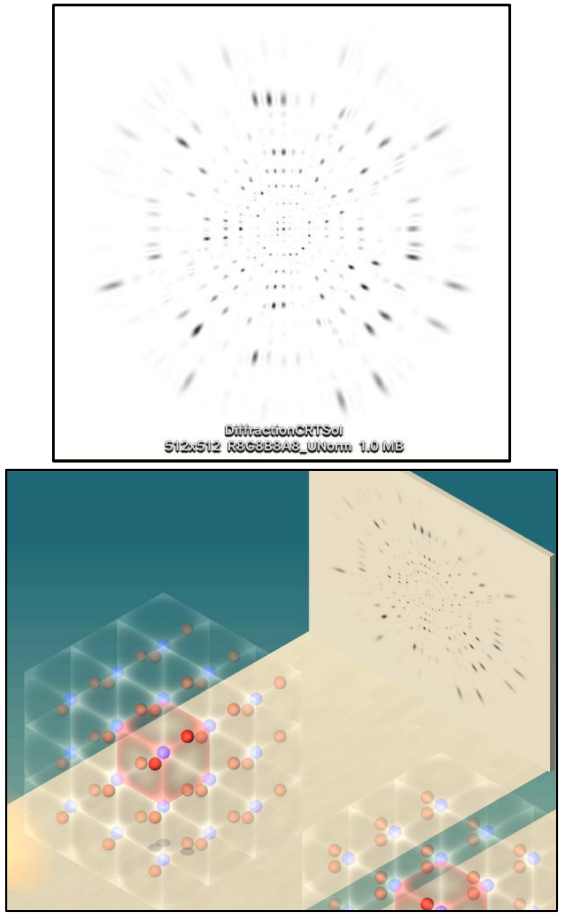
$$K = \left(\frac{\sin(\pi(L+1)(\vec{a}\cdot\vec{s}))}{\sin(\pi(\vec{a}\cdot\vec{s}))}\right)^2 \left(\frac{\sin(\pi(M+1)(\vec{b}\cdot\vec{s}))}{\sin(\pi(\vec{b}\cdot\vec{s}))}\right)^2 \left(\frac{\sin(\pi(N+1)(\vec{c}\cdot\vec{s}))}{\sin(\pi(\vec{c}\cdot\vec{s}))}\right)^2$$

Diffraction pattern in crystal mode

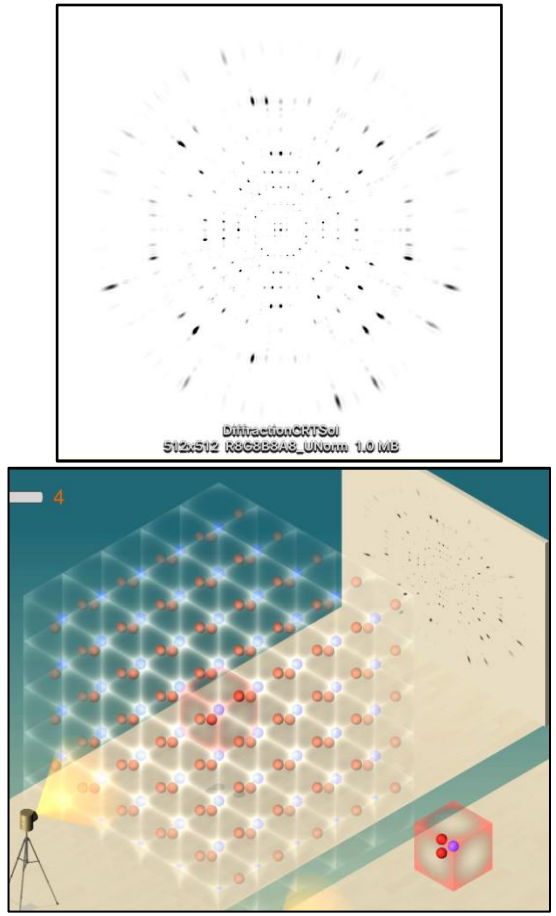
$L = M = N = 0,$
$1 \times 1 \times 1$



$L = M = N = 2,$
$3 \times 3 \times 3$



$L = M = N = 4,$
$5 \times 5 \times 5$



Eurographics

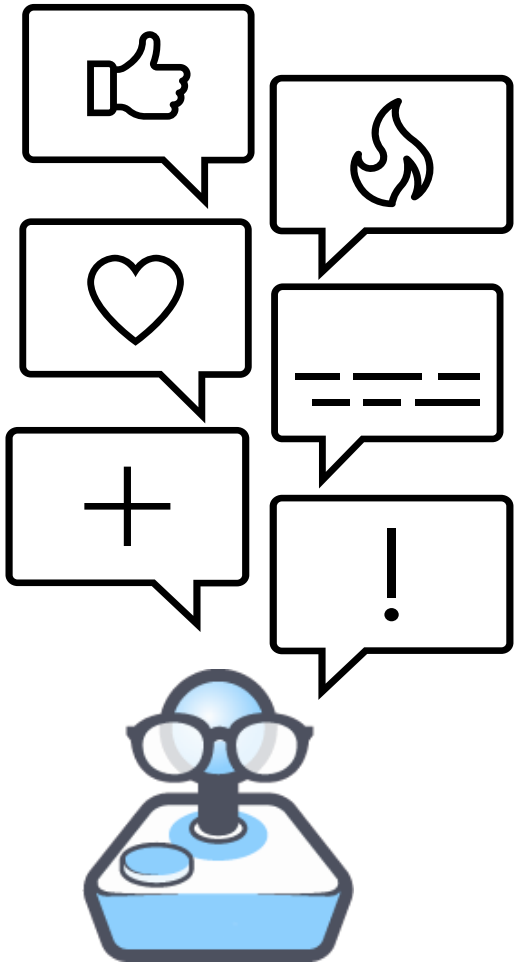
Eurographics
Italian Chapter

STAG

Public Dev-Log

The Independent Game Source:

https://tinyurl.com/TigSource



Eurographics

Eurographics
Italian Chapter

STAG

Thank you for your attention

Game Design considerations

It is possible to evaluate a level difficulty enumerating its degree of freedom.

	2D	3D
Molecule mode NOT active	$n \times 2$	$n \times 3$
Molecule mode active	$n \times 1$	$n \times 2$

1	Very easy
2	
3	
4	
5	
6+	Very hard

n = atoms number excluding the pivot